

1. Introduction and Objectives

The development of autonomous vehicles is an interdisciplinary challenge that requires the seamless integration of mechanical engineering, electronic hardware, and intelligent control algorithms. This report details the engineering process of an autonomous prototype vehicle developed by Team D5 at Hochschule Hamm-Lippstadt. The primary focus of this project was to design, construct, and program a smart car capable of independent navigation and dynamic obstacle handling.

The core objectives of the project were defined as follows:

- **Autonomous Navigation:** The vehicle must successfully follow a predefined path autonomously using infrared (IR) sensor feedback.
- **Early Obstacle Detection:** The system must recognize obstructions in its path early by employing ultrasonic sensors.
- **Dynamic Avoidance and Recovery:** Upon detecting an obstacle, the vehicle must execute an avoidance maneuver, recover the line, and continue its operation without manual control.

To achieve these objectives, the project utilized a comprehensive systems engineering methodology. This approach progressed from initial SysML modeling and simulated testing to 3D mechanical design, software implementation, and iterative physical validation.

2. System Design and Engineering Approach

The autonomous vehicle was developed using an integrated mechatronic design process covering requirements, system modeling, mechanical design, electronics, control software, simulation, and physical validation. The main objectives were autonomous black-line following, obstacle detection and avoidance, line reacquisition, and stable vehicle motion.

2.1 System Modeling

Requirements, use case, activity, sequence, and package diagrams were created to describe the system from complementary viewpoints. The requirements diagram defined and traced the main functions to the corresponding hardware, while the activity diagram described the operational logic. The remaining diagrams supported functional interaction, system organization, and component communication.

The requirements model decomposed the overall autonomous-driving objective into line detection, obstacle detection, motion control, speed optimization, and obstacle avoidance. Two infrared sensors were assigned to line tracking, two ultrasonic sensors to obstacle detection, and the motor-control subsystem to the two drive motors. This provided clear traceability between system requirements and physical implementation.

2.2 Engineering Implementation

The activity model defined the main control sequence: initialize sensors, read sensor data, follow the line, detect an obstacle, execute an avoidance maneuver, reacquire the line, and resume normal operation. This logic was implemented in software using modular classes for the motors, infrared sensors, ultrasonic sensors, and vehicle controller, together with explicit states for obstacle handling and recovery.

Mechanical development was performed in SolidWorks using a component-by-component approach before assembling the complete vehicle model. This allowed the positions of the motors, wheels, sensors, controller, and supporting parts to be checked before physical integration. The design was refined iteratively because sensor placement, vehicle geometry, motor response, and control parameters directly affected overall performance.

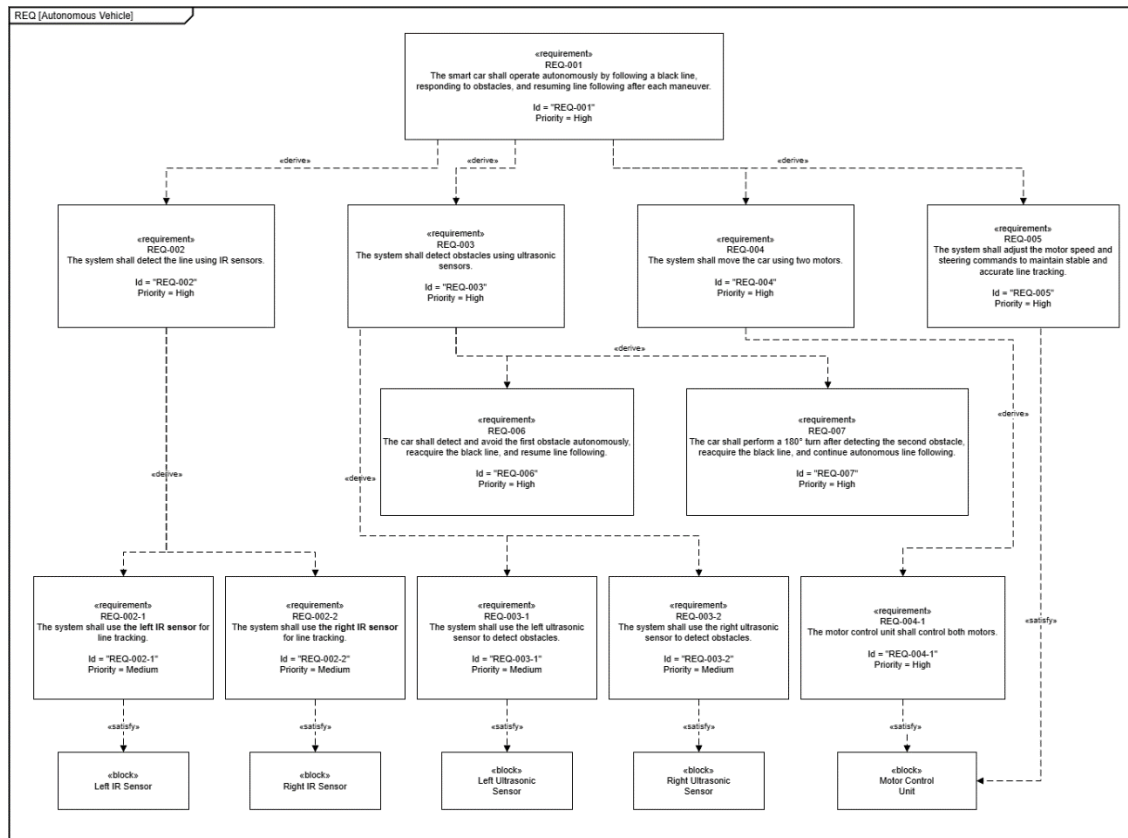


Fig. 1. Requirements model linking the autonomous-driving objectives to line sensing, obstacle sensing, motor control, and speed control.

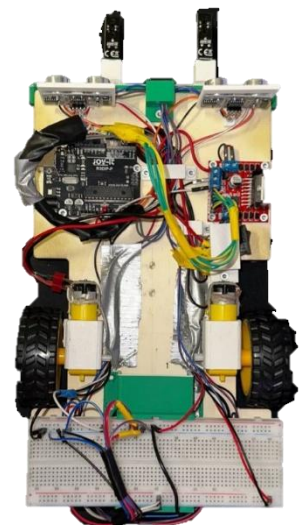
3. Hardware and Software Implementation

3.1 Hardware Integration

The autonomous prototype car was controlled by an Arduino-based microcontroller. Its main hardware components included a breadboard, two motor-driver units, two IR sensors, two ultrasonic sensors, and a battery. The motor drivers controlled the speed and direction of the left and right motors, while the breadboard supported the electrical connections and power distribution.

The two IR sensors were installed at the front and close to the ground to distinguish between the black line and the white track surface. The ultrasonic sensors were also positioned at the front to measure the distance to obstacles. Using two ultrasonic sensors increased the detection area and allowed objects approaching either side of the car to be recognized.

Custom component holders were designed using SolidWorks. Their positions were selected according to the function of each component and the available space on the chassis. The holders secured the sensors while allowing their positions to be adjusted. This arrangement also improved cable management: each wire was routed toward its corresponding component, reducing tangled or overlapping connections and making the electrical system easier to assemble, inspect, and troubleshoot.



3.2 Software Design and Control Logic

The vehicle's control software was developed in C++ utilizing an object-oriented architecture to ensure modularity and ease of troubleshooting. Dedicated classes (Motor, IRSensor, and UltrasonicSensor) were constructed to abstract the low-level pin manipulations and standardize the interface between the hardware and the main control logic.

Prototyping Project Team D5

The core autonomous loop operates on a non-blocking framework, utilizing the microcontroller's internal timer (millis()) rather than standard delay functions. This allows the system to continuously poll the ultrasonic sensors at 30-millisecond intervals without interrupting the real-time line-following adjustments.

During standard operation, the Car class governs navigation. The system reads the track surface via the dual IR sensors; when both detect the white surface, the vehicle drives forward. If one sensor detects the black line, the software dynamically adjusts the differential motor speeds to re-center the vehicle, correcting its trajectory.

To handle obstructions, the software implements a Finite State Machine (FSM) controlled by the AvoidState enumeration. When the ultrasonic sensors detect an object within the 18 cm threshold, the system transitions from the IDLE state into an avoidance sequence:

- **STOP_BEFORE_OBSTACLE:** Halts the DC motors to prevent collision.
- **LEFT_ARC_AROUND & RIGHT_ARC_BACK:** Executes carefully timed, differential motor speeds to bypass the object and sweep back toward the original path. A dynamic distance check is integrated here; if the lateral distance to the object drops below 10 cm, the system temporarily sharpens the turn radius to prevent side-swiping.
- **Line Recovery (SEARCH_LINE & RECOVER_LINE):** Instead of relying solely on dead reckoning, the FSM actively polls the IR sensors while spinning. The loop breaks at the exact millisecond a sensor touches the black line, pausing briefly to let the chassis settle before resuming forward motion.

```
enum AvoidState {
    IDLE, STOP_BEFORE_OBSTACLE, LEFT_ARC_AROUND,
    RIGHT_ARC_BACK, SEARCH_LINE, RECOVER_LINE,
    TURN_180 };

void handleObstacleAvoidance() {
    unsigned long elapsed = millis() - avoidTimer;

    switch (avoidState) {
        case STOP_BEFORE_OBSTACLE:
            stop();

            if (elapsed > STOP_TIME) {
                avoidState = LEFT_ARC_AROUND;
                avoidTimer = millis();
            }

            break;

        case LEFT_ARC_AROUND:
            if (rightSonar.getDistance() > 0 &&
                rightSonar.getDistance() < 10) spinLeft();
            else arcLeft();

            if (elapsed > LEFT_ARC_TIME) {
                avoidState = RIGHT_ARC_BACK;
                avoidTimer = millis();
            }

            break;

        case RIGHT_ARC_BACK:
            arcRight();

            if (elapsed > 150 && blackLineDetected()) {
                delay(250);
                spinLeft();
                while (!blackLineDetected()) {} // Spin
                stop();
                avoidState = RECOVER_LINE;
            }

            break;

        case TURN_180:
            leftMotor.forward(0); rightMotor.forward(65);

            if (elapsed > 1400 && blackLineDetected()) {
                stop();
                avoidState = RECOVER_LINE;
                isTurning = false;
            }

            break;
    }
}
```

Additionally, to fulfill a bonus objective, a dual-obstacle tracking mechanism was implemented. The software maintains a persistent counter for detected obstacles. To ensure a single object is not counted multiple times during a maneuver, a two-second cooldown is enforced between valid detections. Once the system registers a second distinct obstacle, it triggers a TURN_180 state, bypassing the standard avoidance arc to spin the vehicle completely around and automatically recover the line in the opposite direction.

4. Simulation and Testing Results

4.1 Circuit Simulation

The electronic architecture was first modeled in TinkerCAD to verify the main connections and controller interfaces before complete physical testing. The model included an Arduino Uno, an L293D motor driver, two DC motors, two infrared sensors, two HC-SR04 ultrasonic sensors, and the power circuitry.

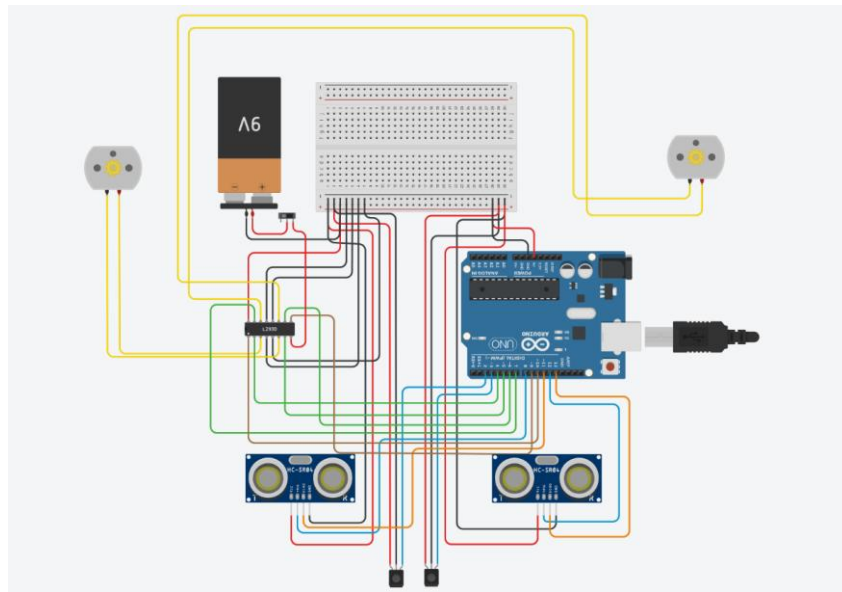


Fig. 2. TinkerCAD circuit model is used to verify the controller, sensors, motor driver, and drive motors.

Parameter	Value	Purpose
Battery supply	9 V	External circuit supply
Controller / sensor logic	5 V	Arduino and sensor operating level
Obstacle detection threshold	≤ 18 cm	Triggers avoidance sequence
Close-obstacle correction	< 10 cm	Applies stronger steering correction

Table 1. Key electrical and distance parameters used in the implemented system.

4.2 Physical Testing

Testing was divided into two main tasks. In Task 1, the vehicle was required to follow the black line autonomously using the two infrared sensors. Motor speed and steering parameters were tuned through repeated runs to improve tracking stability and reduce excessive oscillation. Vehicle speed was determined experimentally from travelled distance and elapsed time.

In Task 2, an obstacle was placed in the vehicle path. When the measured distance reached the defined threshold, normal line following was interrupted and the vehicle executed a programmed avoidance sequence: stop, move

around the obstacle, return toward the original path, detect the black line again, and resume line following. A dedicated search-and-recovery state was included if the line was not immediately required.

The evaluation therefore focused on three performance indicators: line-following stability, vehicle speed, and successful obstacle avoidance followed by line reacquisition. Together, circuit simulation and physical testing provided progressive validation from subsystem integration to complete autonomous operation.

5. Challenges Encountered and Solutions

5.1 IR Sensor Position and Line-Following Method

The first challenge resulted from the mechanical design of the IR sensor holders. The holders were mounted inside an adjustable slider at the front of the chassis. Although the slider allowed their horizontal positions to be changed, it placed the sensors slightly too far above the ground. This made it difficult to position both sensors directly over the black line and obtain consistent readings.

To solve this problem, the control method was changed from following the black line directly to following the surrounding white surface. When both sensors detect white, the car moves forward with the black line centred between them. If one sensor reaches the black line, the program corrects the direction. This method improved stability and allowed the car to move faster when properly centered.

5.2 Speed and Detection-Distance Optimization

Choosing an appropriate driving speed was another challenge. A high speed produced good performance on straight sections, but the car sometimes left the track during sharp turns because the system could not correct its direction quickly enough. Different motor speeds were therefore tested on straight sections, normal curves, and sharp turns. The final values provided a suitable balance between speed and steering response.

The ultrasonic detection distance also required calibration. A large range caused the car to react to nearby walls or people beside the track, whereas a very short range caused late detection and left insufficient space for avoidance. After repeated testing, the distance was set to 18 cm. This reduced false detections while still providing enough space to begin the avoidance maneuver. A sharper correction was added when the distance fell below 10 cm.

5.3 Obstacle Avoidance and Line Recovery

Several avoidance methods were tested. One early approach used a sequence of timed left and right movements, while another produced a V-shaped path. These methods involved sharp direction changes and were inconsistent because small differences in motor speed, starting position, and battery level affected the path. The car could therefore collide with the obstacle while passing it or while returning to the line.

The final solution used a smoother arc-based maneuver. The car stops, follows a left arc around the obstacle, and then follows a longer right arc toward the track. Different speeds are applied to the two motors to form each curve. When the line is detected again, the car crosses it slightly and rotates until the correct position is recovered.

These improvements were achieved through continuous testing and observation. Each result was analyzed to determine whether the problem originated from the mechanical design, sensor configuration, motor speed, or software logic. This iterative process significantly improved the reliability and consistency of the prototype.

6. Conclusions and Future Improvements

6.1 Conclusions

The project successfully yielded a fully functional autonomous prototype vehicle by integrating a custom 3D-printed mechanical structure, sensor arrays, an Arduino logic controller, and motor drivers into a single cohesive unit. Through structured system engineering and continuous testing, the initial theoretical models were effectively translated into a reliable physical system.

A primary learning outcome of this project was understanding the critical interplay between mechanical component placement, software logic, and speed optimization. Physical testing directly influenced the final vehicle behaviour; for instance, evaluation results prompted a shift in the line-following concept and refined the timing sequence for the arc-based obstacle avoidance maneuvers. Ultimately, the vehicle consistently demonstrated the ability to track a path, dynamically maneuver around obstacles, and recover its route autonomously.

6.2 Future Improvements

While the current prototype meets all primary project requirements, several areas have been identified for future enhancement to increase the vehicle's speed, precision, and overall reliability:

- **PID Control Implementation:** Upgrading the steering logic to a Proportional-Integral-Derivative (PID) controller would allow for smoother trajectory corrections and reduce oscillations during active line following.
- **Expanded Sensor Array:** Incorporating additional IR sensors would improve surface resolution, facilitating better and faster line recovery after an avoidance maneuver.
- **Hardware Upgrades:** Integrating an enhanced motor control driver would provide finer speed regulation and improved handling of the DC motors.
- **Custom Printed Circuit Board (PCB):** Replacing the prototype breadboard with a custom-specified PCB would significantly improve wiring reliability, reduce electrical noise, and condense the electronic footprint of the project.